



Beacon Night Shift

The on-call agent that fixes the 3 AM page with your voice —
and does not wake you the second time.

WHAT ON-CALL ACTUALLY COSTS

53_{min}

median time to recovery, industry-wide

12 % better in five years — against 3× the monitoring spend

74%

of DevOps engineers report burnout

on-call load is the leading indicator; seniors leave first

\$5.6k

per minute of downtime, Gartner baseline

and on a team of four there is nobody to escalate to

WHO THIS IS FOR

**A backend engineer on a team of one to five,
running production on AWS for users in another
time zone.**

No follow-the-sun rotation.
There is no second shift to
escalate to.

A small, repeating failure
surface: a rule dropped by a
deploy, a service that needs a
redploy.

No budget for an SRE
platform, and nobody to run
one.

LIVE, NOT A MOCK-UP

Speak over the read-back, and **the fix is withdrawn.**



The screenshot shows a dark green rounded rectangle with a circular icon on the left. The icon has a grey outer ring, a yellow inner ring, and a black center. To the right of the icon, the text reads: "Beacon is speaking — talk to interrupt", "heard: 'No, wait, stop.'", a checkbox labeled "noisy room", and "Interrupt". Below this, a white box contains the text: "BEACON 00:15:24 A security group rule that the golden snapshot says should exist is missing, so the service can no longer — INTERRUPTED".

Beacon is speaking — talk to interrupt
heard: "No, wait, stop."
☐ noisy room
Interrupt

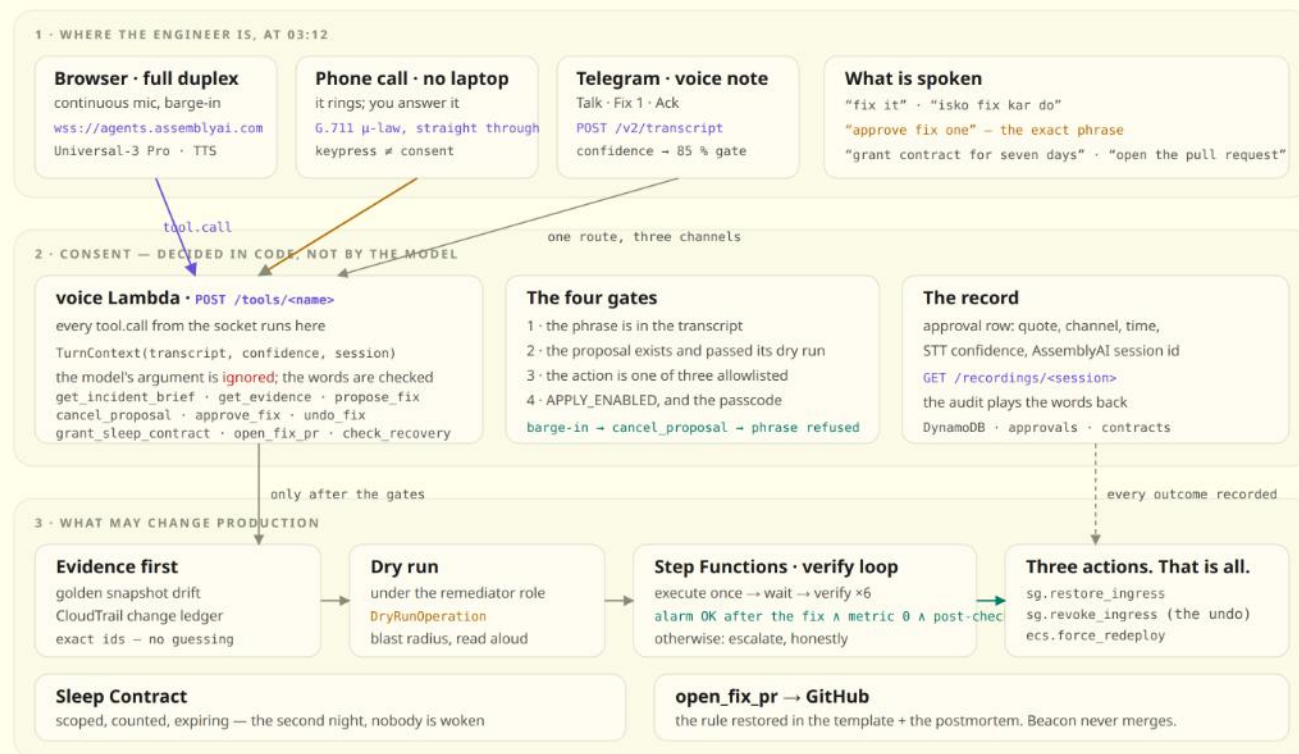
BEACON 00:15:24 A security group rule that the golden snapshot says should exist is missing, so the service can no longer —
● INTERRUPTED

A real run on the deployed account: the read-back cut mid-sentence, **INTERRUPTED**, `propose_fix · dry run PASSED`, and the console note — "Fix 1 withdrawn — you spoke over the read-back, so nothing was applied." The approval phrase stops working until it is proposed again.

Nine tools on one socket. Consent is decided in code.

Beacon Night Shift — the voice path

Nine tools on one socket. The model decides whether to call one; code decides whether the call is allowed.



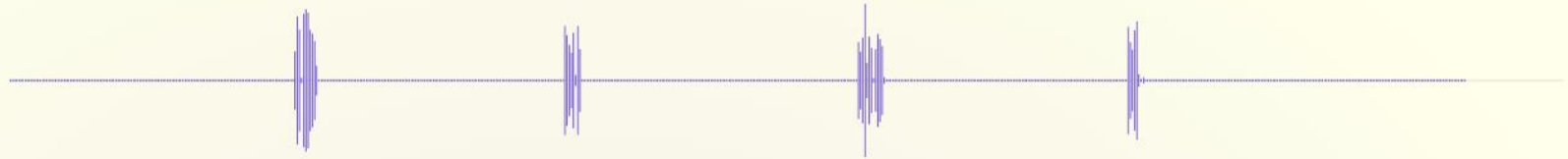
The Voice Agent API is the mouth and the ears; every **tool.call** returns to the browser, which runs it on our Lambda with the transcript the API produced.

THE PHONE CALL

No laptop. It rings, and you answer it.

Channel 1 — you

8 kHz · G.711 μ -law



Channel 2 — Beacon

AssemblyAI · nine tools



you said “fix”

approve_fix — the phrase is on **your** channel. Beacon read it back twice; a read-back is not consent.

CHECKED

69.6 s · get_evidence → propose_fix → approve_fix → check_recovery · the recording is two channels, so who approved what is a fact about the file

WHY THIS NEEDED THIS GENERATION OF MODELS

- 01 Consent has to rest on words an engineer actually said — half asleep, in two languages. A keypad cannot say which fix on which resource.
- 02 An interruption has to be an event the application can act on. Speaking over the read-back withdraws the fix.
- 03 Nine tools have to be callable mid-sentence, with each result feeding the next thing said.

TESTS THAT SPEAK

Real audio · the Voice Agent API · the real tools

approve

the exact phrase applies the fix and the verify loop starts

approve_fix

wrong_phrase

“yes, do it” changes nothing — agreement is not consent

propose only

barge_in

speaking over the read-back **withdraws** the proposal

cancel_proposal

hinglish

“isko fix kar do” reaches the same tool

propose_fix

keypad

on a phone, a **keypress** may acknowledge — it may never approve

no consent tool

5 of 5 down a telephone line, at 8 kHz, through the real API — plus six more in the browser. Each one speaks, then asserts on what was heard, which tools ran, and how the incident ended.

It went 2/5 to 5/5 in an afternoon. Every step was a fault it found: two in the test harness that looked exactly like the product, and two in the product — a key-term list that primed the wrong fix number, and an attestation that could not read a phrase the transcriber had split into three.

MARKET

TAM

\$4.8 B

incident management software, 2026 → \$12.99 B by 2035

SAM

\$1.2 B

alert response and auto-remediation — PagerDuty alone
books \$493 M

SOM

\$125 M

small AWS teams with a one-person rotation, at list price

120,000 teams of ten or fewer × 3 responders × \$29/month · assumptions stated, argue with them

HOW IT EARNS

\$29

per responder, per month
PagerDuty is \$21–41 a seat

\$2

per **verified** remediation —
charged only when the fix was
applied *and* CloudWatch agreed
we earn nothing from noisy alerts

\$15k

per year, self-hosted
three stacks in your own account;
no data leaves

BUILT SOLO, IN TEN DAYS

SHIPPED

176 commits

406 tests

3 channels, one socket

live URL, open during judging

PROVEN, NOT CLAIMED

socket killed mid-approval → **nothing ran**

barge-in → **fix withdrawn**

a keypress → **never consent**

each one is a test or a spoken run

NEXT

WhatsApp as the fourth channel

a carrier number, always on

contract policies for teams

a second cloud